ComicShelf

Purpose: To help users track the physical comics they own so they can, know what they already have, track where each comic is stored (box/shelf), track condition and value, quickly look up an issue and update its status. This will be a very lite inventory tracker for comics for collectors.

Potential Users: The primary target for this project will be comic collectors that may have amassed huge collections or starter collectors that are just beginning their collection journey. Another user that is potential is hobbyists who buy single issue or just trade paperbacks, sharing/sending comics to friends and to remember who has what.

REST API routes used by React Native app

1. GET /api
    a. Use: Load and display the full comic collection
        i. App Startup: fetch all comics and show in a FlatList
        ii. After any create/update/delete: re-ftech to keep the UI synced
    b. UI:
        i. Main Screen: Comic List
        ii. Search bar to filter locally (optional if I have time to implement)
        iii. Pull to refresh / refresh button (optional if I have time to implement)
2. POST /api
    a. Use: Adds new comics the user owns
    b. UI:
        i. Add comic form with inputs:
            1. TextInput: Series Title
            2. TextInput: Issue Number
            3. TextInput: Publisher
            4. Picker: Condition (MINT/F/G/P and so on)
            5. TextInput: Storage Location (Box1, Box2, Shelf1, Shelf A)
            6. TextInput: Estimated Value
        ii. Button: Add to collection
3. GET /api/:id
    a. Use: View a comic full details
    b. UI:
        i. Details screen:
            1. Shows all the fields and actions (Edit/Delete)
4. PUT /api/:id
    a. Use: Update a comic's details(whenever it moves places, price updated, mark as lentout, and so on)
    b. UI:
        i. Edit comic screen
            1. Button: Save Changes
5. DELET api/:id
    a. Use: Remove One comic from the collection

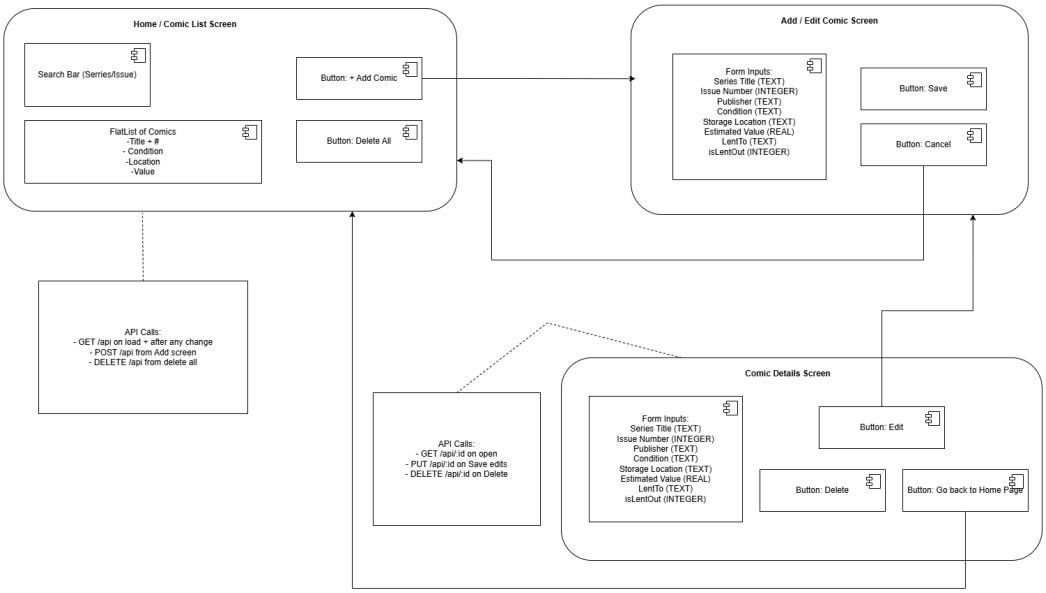
    b. UI:
        i. Delete button on the details screen
        ii. Confirmation button (optional)
6. DELETE /api
    a. Use: Clear the entire collection (starting fresh or somehow collection is sold)
    b. UI:
        i. Settings screen
            1. Button: Delete Entire Collection
            2. Confirmation Button NEEDED <- important to make sure the user doesn't accidently delete their entire collection

Collection item fields and types (SQLite)

Field	Type	Example	Use
id	INTEGER(PK, auto-increment)	21	Unique record id
seriesTitle	TEXT	“Absolute Batman”	Core identifier
issueNumber	INTEGER	12	Numeric Issue tracking
publisher	TEXT	DC	Helps for filter
condition	TEXT	Near Mint	Something Collectors care about
storageLocation	TEXT	Shelf A	Finding where it is physically
estimatedValue	REAL	4.99	Price Tracking

If I have time to add I would like to add LentTo TEXT or isLentOut INTEGER so I can have a way to track down if a user let someone borrow the comic and who borrowed it. (optional if I have time to implement)

UI Mockup:


Home / Comic List Screen
Search Bar (Serries/Issue)
Button: + Add Comic
FlatList of Comics
- Title + #
- Condition
-Location
-Value
Button: Delete All
Add / Edit Comic Screen
Form Inputs:
Series Title (TEXT)
Issue Number (INTEGER)
Publisher (TEXT)
Condition (TEXT)
Storage Location (TEXT)
Estimated Value (REAL)
LentTo (TEXT)
isLentOut (INTEGER)
Button: Save
Button: Cancel
API Calls:
- GET /api on load + after any change
- POST /api from Add screen
- DELETE /api from delete all
Comic Details Screen
API Calls:
- GET /api/:id on open
- PUT /api/:id on Save edits
- DELETE /api/:id on Delete
Form Inputs:
Series Title (TEXT)
Issue Number (INTEGER)
Publisher (TEXT)
Condition (TEXT)
Storage Location (TEXT)
Estimated Value (REAL)
LentTo (TEXT)
isLentOut (INTEGER)
Button: Edit
Button: Delete
Button: Go back to Home Page